

Branch Prediction vs Memory Locality in Sorting

The Branchless Lomuto Partition

CMPT 295 Mini-Project — Nathan Omana — August 10, 2026

1 Introduction

Two sorts that are both $O(n \log n)$ can differ by up to roughly $10\times$ in performance. The reason for this is not that one does more work. It is because one produces branch mispredictions that stall the CPU pipeline.

Question: a branchless Lomuto partition removes data-dependent branches by using a conditional move instruction (`cinc` on ARM64, `cmov` on x86). It pays for this with more instructions per element. Where does a branchless sort perform better, where does it perform worse, and why?

2 The Four Implementations

All four sort `int32_t` arrays in place. All use the same interface: `void sort(int32_t *a, size_t n)`.

`insertion_sort` with worst case $O(n^2)$. Limited to $n \leq 64K$.

`quicksort_branchy` — Lomuto partition with a branchy swap:

```
for (size_t j = lo; j < hi; j++) {
    if (a[j] < pivot) {        // data-dependent branch
        swap(&a[i], &a[j]);
        i++;
    }
}
```

Pivot selection is median-of-three. On random input the pivot lands near the true median, so approx. 50% of elements pass the test. The branch outcome looks random to the predictor, which then mispredicts roughly half the time. Each misprediction costs about 15–20 cycles on a modern CPU.

`quicksort_branchless` — the key principle is that swapping does no harm when `a[j]` is greater than or equal to the pivot. Both elements are already in the region at or above the pivot, so the partition is unchanged. The swap can therefore run unconditionally. Only the index update stays conditional:

```
for (size_t j = lo; j < hi; j++) {
    swap(&a[i], &a[j]);        // always -- safe either way
    i += (a[i] < pivot);      // compiles to cinc/cmov -- no branch
}
```

`i += (comparison)` compiles to a single `cinc` (ARM64) or `cmov` (x86). The CPU evaluates it once per iteration with no pipeline flush. The disadvantage is that the swap now runs for every element, including when it is not needed.

`stdlib_qsort` — wraps the platform C library's `qsort`. Its comparator is a function pointer, so the compiler cannot inline it. Every comparison becomes an indirect call, costing roughly 5–15 extra cycles. This is a cost of API design, not of the algorithm.

3 How Measurement Was Conducted

Machine. Apple MacBook Air, Apple M2 (ARM64). L1d is 128 KB per performance core, or about 32K `int32` elements. L2 is 16 MB shared across the four performance cores, about 4M elements. There is no conventional L3. Memory is 8 GB unified LPDDR5 at roughly 100 GB/s.

SSH access to the SFU Linux lab machine (i9-14900) stalled in the week leading up to this report, so all measurements were taken locally. The argument still holds on ARM64, but the cache sizes are different. Valgrind also does not run on Apple Silicon. That means no branch misprediction counts were obtainable, and the branch argument below rests on controlled timing comparisons rather than direct counting.

Compiler. Apple clang 17.0.0, flags `-std=c11 -Wall -march=native`. Results are reported at `-O2`.

Timing. `clock_gettime(CLOCK_MONOTONIC)`. I did not use `rdtsc`. The timestamp counter runs at a fixed reference frequency rather than the core frequency, so its counts do not correspond to cycles.

Protocol. Seven measurements per configuration. Measurement 0 is discarded as warm-up and the median of measurements 1–6 is reported. The array is restored from a pristine copy before each trial, with the copy outside the timed region. The output is checksummed so the optimizer cannot delete the sort. Correctness is checked for sortedness and as a multiset against the input. Inputs come from a seeded xorshift64 generator [2].

Distributions. *random* (uniform values) gives about 50% misprediction. *sorted* (ascending) makes branches perfectly predictable. *reverse* (descending) is predictable in the opposite direction. *partial* (95% sorted, 5% shuffled) is the transition case. *dupes* (about 10 distinct values) stresses Lomuto's equal-key weakness.

4 Results

4.1 Was the compiler covertly converting branchy to branchless?

This is critical. If clang compiles the guarded swap to a `cinc` on its own at `-O2`, then both branches of the experiment contain identical code and the experiment is impossible.

Answer: no. The branchy loop keeps a real conditional branch. The branchless loop has none and uses `cinc` instead.

```

; BRANCHY
ldr w10, [x19, x9, lsl #2] ; load a[j]
cmp w10, w8                ; vs pivot
b.ge LBB2_10               ; <== BRANCH
ldr w11, [x19, x22, lsl #2] ; load a[i]
str w11, [x19, x22, lsl #2] ; a[i]=a[j]
ldr w10, [x19, x22, lsl #2] ; a[i]=a[j]
str w10, [x19, x9, lsl #2] ; a[j]=old a[i]
ldr w10, [x19, x22, lsl #2] ; reload
str w11, [x19, x9, lsl #2] ; a[j]=old
add x22, x22, #1           ; i++
                                ; cinc x22, x22, lt ; i++ if less, NO BRANCH
; BRANCHLESS
ldr w10, [x19, x22, lsl #2] ; load a[i]
ldr w11, [x19, x9, lsl #2] ; load a[j]
str w11, [x19, x22, lsl #2] ; a[i]=a[j]
str w10, [x19, x9, lsl #2] ; a[j]=old a[i]
ldr w10, [x19, x22, lsl #2] ; reload
cmp w10, w8                ; vs pivot
cinc x22, x22, lt          ; i++ if less, NO BRANCH

```

`cinc` is the ARM64 counterpart of x86 `cmov`. The CPU produces either `x22` or `x22 + 1`, with no branch and no pipeline flush. The difference between the two versions is real, and it survives at both `-O2` and `-O3`.

4.2 Vector instructions

No loop in `sorts.c` is vectorized. Apple’s `gcc` is really `clang`, so the GCC flag `-fopt-info-vec` is not available; `-Rpass-missed=loop-vectorize` gives the equivalent report:

```

sorts.c:33  insertion inner loop    -- not vectorized
sorts.c:51  branchy partition loop -- not vectorized
sorts.c:104 branchless partition loop -- not vectorized

```

This is expected. The partition loop carries a dependency on `i`: the next store address is unknown until the current comparison updates the index, and the store addresses are themselves data-dependent. Consecutive iterations cannot be packed into one NEON register operation. This is why vectorized sorts use sorting networks instead of vectorizing a partition. The M2’s 128-bit NEON registers go unused in the hot loop, so the instruction-level difference between the two variants is just `b.ge` versus `cinc`.

4.3 Timing: random input across all sizes

Times are normalized by $n \log_2 n$. A perfect $O(n \log n)$ sort is therefore a flat line, and any bump shows a real machine effect.

n	branchy	branchless	Result
8K	4.38 ns	5.11 ns	branchy 17% faster
32K	5.18 ns	4.69 ns	branchless 9% faster ← only crossover
128K	4.76 ns	4.92 ns	branchy 3% faster
512K	4.62 ns	4.90 ns	branchy 6% faster
2M	4.70 ns	4.87 ns	branchy 4% faster
8M	4.41 ns	4.38 ns	tied
32M	4.09 ns	4.17 ns	tied
64M	4.21 ns	4.25 ns	tied

Table 1: Normalized time, random input, `-O2`. Run-to-run spread is within about 8% of the median. Single differences below that are not individually significant, but the sign is consistent across sizes.

Inside L1d ($n < 32K$). Branchy is faster. The branchless loop swaps every element whether it needs to or not. At this size that extra work costs more than the mispredictions it removes.

At $n = 32K$, the L1d boundary. The only point where branchless wins, by 9%. Extra pressure at the spill point lengthens each stall, which briefly makes removing the branch worthwhile.

In the L2 range (128K–2M). Branchy recovers and is 3–6% faster. The branchless overhead dominates again.

Past L2 ($n \geq 4M$). The two are equal to within noise.

`stdlib_qsort` was 1.22–1.57× slower than branchy at every size tested. The indirect call cost appears regardless of array size.

One thing the table does not show is a cache staircase. Normalized time does not rise as the array outgrows L1 and L2. It stays flat, and even drops slightly at the largest sizes. Quicksort turns out to be cache-friendly. Its partition pass is a sequential scan, which the prefetcher handles well. Its recursion also means that after a few levels it works on subarrays that fit in cache. Only the top levels touch the whole array, and they are a small fraction of total work.

4.4 Input distribution: the branch prediction signal

Sorted. Branchy is 7.7–10.9× faster. Every `a[j]` compares the same way against the pivot, so there are no mispredictions. Branchy pays nothing for its branch. Branchless still swaps every element regardless. The extra work is entirely wasted.

Reverse. Branchy roughly 2× faster, for the same reason.

Partial (95% sorted). Branchy roughly 2× faster. 5% disorder does not produce enough mispredictions to matter.

Random. Effectively tied. This is where branchy’s misprediction cost is largest and where branchless comes closest.

Dupes (about 10 distinct values). Both variants degrade badly. Branchless is about 8× slower than branchy at $n = 32K$. The cause is Lomuto’s equal-key weakness: the strict `<` test sends every element equal to the pivot into the left half, which produces near-quadratic partitions. Both variants were capped at $n = 32K$ elements here to keep runtimes reasonable. This is a property of the partition scheme, not of branch prediction.

Misprediction cost without Cachelgrind. Branchy runs at about $0.85 \text{ ns}/n \log_2 n$ on sorted data and about 4.6 on

random data. That is a $5.4\times$ slowdown. The same binary ran the same instruction sequence over the same array size in both cases, so misprediction is the dominant difference between them.

4.5 Optimization level

Level	$n = 128K$		$n = 8M$	
	branchy	branchless	branchy	branchless
-00	24.0 ms	25.5 ms	2217 ms	2104 ms
-01	10.0 ms	10.5 ms	849 ms	919 ms
-02	10.1 ms	10.2 ms	834 ms	866 ms
-03	10.7 ms	10.4 ms	891 ms	916 ms

Table 2: Absolute time by optimization level, random input.

The big step is -00 to -01, about $2.4\times$ for both versions. At -00 every variable lives on the stack and every expression goes through memory. At -01 the compiler holds `i`, `j` and `pivot` in registers and drops the redundant stores. This matters because the inner loop runs $n \log_2 n$ times. -02 and -03 add very little, since the loop was already efficient at -01.

There is an interesting result at -00. At $n = 8M$, branchless (2104ms) beats branchy (2217ms). At that level `i += (a[i] < pivot)` compiles to a branch rather than a `cinc`, so the branchless version is not actually branchless without optimization enabled. The `cinc` appears only once the compiler is allowed to optimize.

5 What I Learned

5.1 Reasoning about the results

Why branchy wins inside L1d. The branchless loop pays for every element with an unconditional swap. At small sizes that cost is larger than the misprediction penalty it avoids.

Why sorted input is the clearest case. Predictable branches plus unconditional swaps means branchless loses badly. The $7.7\text{--}10.9\times$ ratio is structural, not noise. Branchless does the same work regardless of input. Branchy only swaps when it has to.

stdlib_qsort overhead. The function-pointer comparison costs the same on every comparison. That includes sorted input, where there are no mispredictions to pay for. The $1.22\text{--}1.57\times$ slowdown stays roughly constant across all inputs because it is call overhead, not branch overhead.

-00 to -01 is the important optimization. Register allocation turns the hot loop from memory-bound on its own working variables into compute-bound.

5.2 Predictions: what I got right and wrong

	Prediction	Outcome
P1	branchless loses on sorted input	HELD. $7.7\text{--}10.9\times$ slower. No mispredictions exist to remove, so the extra swaps are pure overhead.
P2	crossover near the L2 boundary	PARTIALLY HELD. A crossover exists, but at the L1d boundary ($n = 32K$) rather than L2 ($n = 4M$). It happens only once before branchy recovers.
P3	convergence past last-level cache	HELD. Beyond 4M elements the two stay within 2% of each other, all the way to 64M.
P4	compiler turns branchy into branchless at -02	DID NOT HOLD. Apple clang keeps <code>b.ge</code> in the branchy loop at both -02 and -03. <code>cinc</code> appears only in the branchless loop.
P5	stdlib slowest	HELD. $1.22\text{--}1.57\times$ slower than branchy on random input at every size tested, and constant across inputs.
P6	dupes worst for Lomuto	HELD. Both variants go near-quadratic with many duplicates, and the test size had to be capped at $n = 32K$.

5.3 Limitations

No branch misprediction counts. Valgrind and Cachegrind do not work on Apple Silicon. Misprediction cost is inferred from timing rather than measured.

No core affinity. macOS has no `taskset` equivalent, and the M2 has both performance and efficiency cores. Median-of-7 was used to limit the damage. Most variance is within 8% of the median.

ARM64, not x86-64. The misprediction penalty and the code generated for the conditional move both differ on x86-64, which also has an L3 that the M2 lacks. Running the same sweep on x86-64 is the obvious next experiment.

Lomuto is not an optimized partition. Hoare’s scheme is faster, and pdqsort and radix sort are faster still. This project is about branches versus cache, not about sorting throughput.

References

- [1] Bentley, J. (1999). *Programming Pearls*, 2nd ed. Addison-Wesley (Lomuto partition, Column 11). [2] Marsaglia, G. (2003). Xorshift RNGs. *J. Statistical Software*, 8(14).
Sedgewick, R. (1978). Implementing quicksort programs. *CACM*, 21(10), 847–857. [4] Stavenga, G. (2022). Branchless Lomuto Partition. <https://gerben-stavenga.github.io/posts/branchless-lomuto/>